

CSC1031: OOP

Week 3

Overloading, static members

Constructor Chaining

Constructor chaining refers to the process of **calling one constructor from another** within the same class or across a class hierarchy. It allows efficient object initialization by reducing code duplication

Purpose

Code Reuse – Avoid repeating initialization logic in multiple constructors

Avoid Redundancy – Ensures that every constructor calls a central logic to maintain consistency

How `this()` Works

- `this()` is used to call another **constructor of the same class**.
- It **must be the first statement** inside a constructor.

```
class Car {
    String model;
    int year;

    Car() {
        model = "Unknown";
        year = 2000;
    }

    Car(String model) {
        this.model = model;
        year = 2000;
    }

    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
}
```

```
class Car {
    String model;
    int year;

    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    Car() { this("Unknown", 2000); }

    Car(String model) { this(model, 2000); }
}
```

Method Overloading in Java

Method Overloading allows **multiple methods with the same name** but with **different parameters** within the same class. The compiler differentiates them based on their method **signature** (number and type of parameters).

Methods must have the same name but different parameter lists

Overloading is determined by:

- ✅ Different number of parameters
- ✅ Different types of parameters
- ❌ Return type does NOT count for overloading

Java differentiates these methods based on **parameter types**.

```
class MathOperations {  
    void multiply(int a, int b) {  
        System.out.println("Integer multiplication: " + (a * b));  
    }  
  
    void multiply(double a, double b) {  
        System.out.println("Double multiplication: " + (a * b));  
    }  
}
```

Different Number of Parameters

```
class MathOperations {  
    void multiply(int a, int b) {  
        System.out.println("Product: " + (a * b));  
    }  
  
    void multiply(int a, int b, int c) {  
        System.out.println("Product: " + (a * b * c));  
    }  
}
```

The method signature (method name + parameter types) is the same.
Java **cannot distinguish methods by return type alone.**

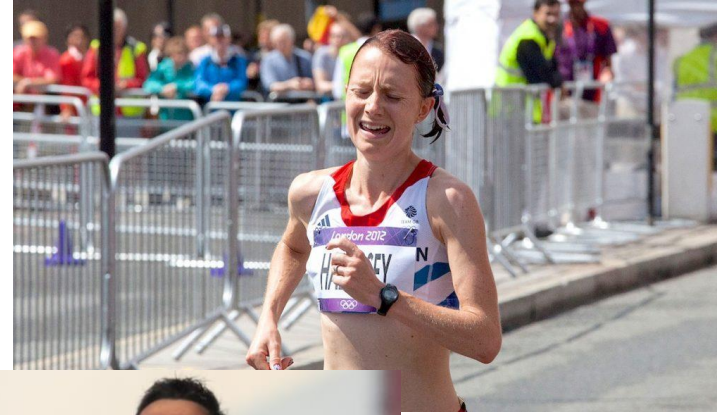
```
class Example {  
    int add(int a, int b) { return a + b; }  
    double add(int a, int b) { return a + b; } // ❌ Compiler error  
}
```

```
class Example {  
    void process(String name, int age) {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
  
    void process(int age, String name) {  
        System.out.println("Age: " + age + ", Name: " + name);  
    }  
}
```

"Jogging first and having lunch afterward"



"Having lunch first and jogging afterward"



Order Matters in Overloading

"Having lunch first and jogging afterward"

"Jogging first and having lunch afterward"

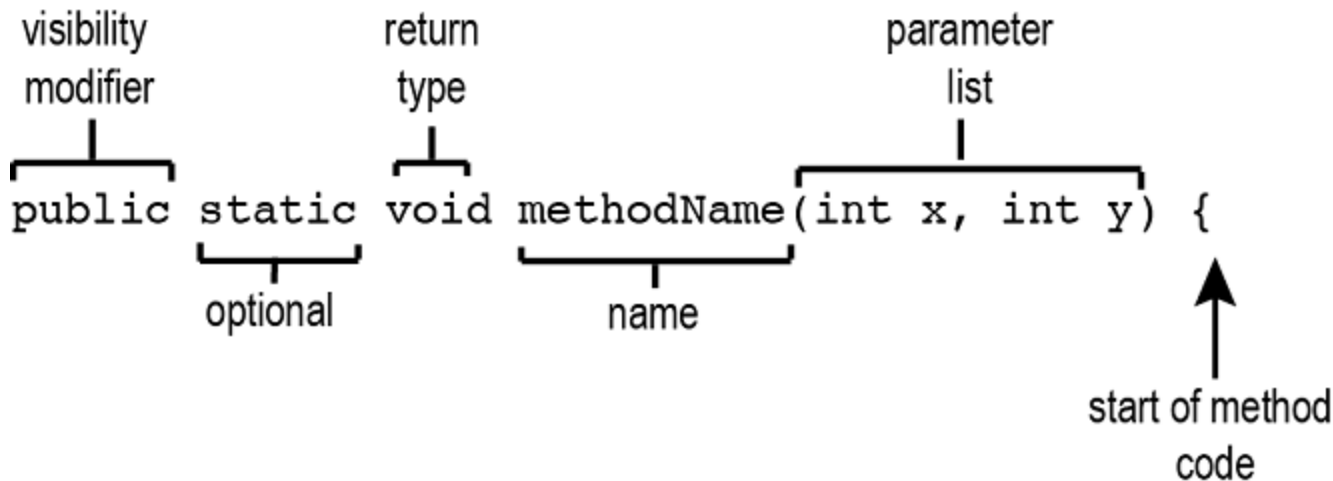
```
class Example {  
    void process(String name, int age) {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
  
    void process(int age, String name) {  
        System.out.println("Age: " + age + ", Name: " + name);  
    }  
}
```

Method Signature

The method signature consists of:

Method Name + Parameter Types (in order)

```
void calculateAnswer(double, int, double, double)
```



Overloading with Type Promotion

Java promotes smaller types to larger types.

Type promotion occurs when a smaller data type is automatically converted into a larger data type to match a method parameter.

Rules of Type Promotion in Overloading

- If an exact match is not found, Java promotes the argument to the next larger type
- `byte` → `short` → `int` → `long` → `float` → `double`
- `char` can be promoted to `int`, `long`, `float`, or `double`
- `Boolean` is NOT type promoted

Example – Overloading with Type Promotion

```
class TypePromotionExample {  
    void show(int a) {  
        System.out.println("Integer method: " + a);  
    }  
  
    void show(double a) {  
        System.out.println("Double method: " + a);  
    }  
  
    public static void main(String[] args) {  
        TypePromotionExample obj = new TypePromotionExample();  
        obj.show(5);    // Calls show(int)  
        obj.show(5.5); // Calls show(double)  
        obj.show(5.0f); // float promoted to double, calls show(double)  
    }  
}
```

What Happens When There's No Exact Match?

- Promotion kicks in when the exact type is missing.
- Java picks the closest, larger compatible type.
- But what happens when multiple promotions are possible?

```
class Test {  
    void display(int a) {  
        System.out.println("int method");  
    }  
  
    void display(double a) {  
        System.out.println("double method");  
    }  
  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.display('A');  
        // 'A' (char) is promoted to int, calls display(int)  
        obj.display(10L);  
        // long is promoted to double, calls display(double)  
    }  
}
```

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

Type Promotion in Ambiguous Cases

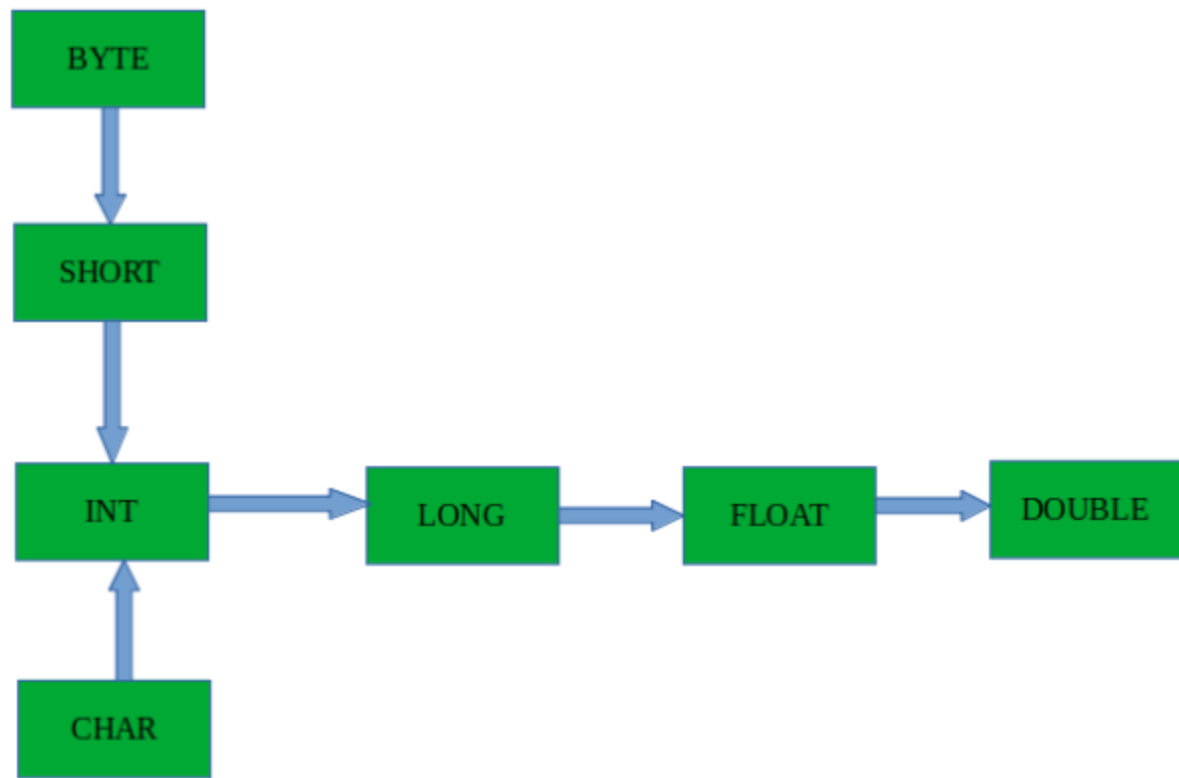
What happens when multiple promotions are possible?

5 (int) has no exact match

It can be promoted to both float and double
Which one will Java choose?

Java prefers the "smallest" possible promotion,
so it calls `display(float)`

```
class Test {  
    void display(float a) {  
        System.out.println("Float method");  
    }  
  
    void display(double a) {  
        System.out.println("Double method");  
    }  
  
    public static void main(String[] args) {  
        Test obj = new Test();  
        // ? Which method will be called?  
        obj.display(5);  
    }  
}
```



Introduction to Varargs (Variable Arguments)

Varargs allows a method to accept **zero or more arguments** of a specified type.

```
void methodName(type... varName) { }
```

Provide **flexibility** in method parameters

Avoid **method overloading** for multiple argument counts

```
void printNumbers(int... numbers) {  
    for (int num : numbers) {  
        System.out.println(num);  
    }  
}
```

Call with any number of arguments:

- `printNumbers(1);`
- `printNumbers(1, 2, 3);`
- `printNumbers();`

Rules and Limitations of Varargs

Only one **varargs parameter** is allowed per method.

Varargs **must be the last parameter** in the method signature

```
void example(int a, String... names) { } //  Valid  
void example(String... names, int a) { } //  Invalid
```

Varargs vs. Arrays

Varargs is essentially treated as an array internally.

However, it provides more **convenience** in method calls without explicit array creation.

Limitation:

- Varargs can lead to **ambiguity** if not used carefully with overloaded methods.

Examples and Best Practices

```
class Calculator {  
    int sum(int... numbers) {  
        int total = 0;  
        for (int num : numbers) {  
            total += num;  
        }  
        return total;  
    }  
  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.sum(1, 2, 3));           // Output: 6  
        System.out.println(calc.sum(5, 10, 15, 20));    // Output: 50  
    }  
}
```

Static

Static Members

- In Java, static members belong to the class rather than instances. This means:
 - Shared across all objects of the class
 - Loaded into memory once at class loading time
 - Can be accessed without creating an object
- **Types of Static Members:**
 - Static Variables – Shared across all instances
 - Static Methods – Can be called without an instance
 - Static Blocks – Executes once when the class loads
 - Static Inner Classes – A nested class that doesn't need an outer class instance

Static Variables – Shared Across Objects

A static variable is common for all objects.

Stored in the class memory, not in individual objects.

All objects share the same static variable `count`.

```
class Counter {
    static int count = 0; // Static variable

    Counter() {
        count++; // Incrementing static variable
    }

    void displayCount() {
        System.out.println("Count: " + count);
    }
}

public class Test {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();

        c1.displayCount(); // Output: Count: 3
        c2.displayCount(); // Output: Count: 3
    }
}
```

Static Methods – Calling Without an Object

Static methods belong to the class, not instances.

Cannot use **this** inside static methods.

Can only access other static members directly.

Static methods can be called using the class name (**MathUtil.square(5)**).

No need to create an instance.

```
class MathUtil {  
    static int square(int num) {  
        return num * num;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Square of 5: " + MathUtil.square(5));  
        // No object needed  
    }  
}
```

Static Blocks – Code That Runs Once

A static block executes once when the class is loaded, before the `main()` method. Used for one-time initialization (e.g., database connections, constants).

Static blocks execute first, even before `main()`.

```
class Example {  
    static {  
        System.out.println("Static block executed.");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Main method executed.");  
    }  
}
```

Output:
Static block executed.
Main method executed.

Static vs. Non-Static Inner Classes

Inner classes are classes declared inside another class. They help in logically grouping code and improving encapsulation.

Two main types of inner classes:

Static Inner Class (Nested Static Class) – Does NOT require an instance of the outer class.

Non-Static Inner Class (Member Inner Class) – Requires an instance of the outer class.

Key difference: A **non-static inner class** is bound to an instance, while a **static inner class** exists independently.

Feature	Static Inner Class	Non-Static Inner Class
Requires Outer Class Instance?	✗ No	✓ Yes
Can Access Outer Class's Non-Static Members?	✗ No	✓ Yes
Can Have <code>static</code> Members?	✓ Yes	✗ No
Memory Allocation	Created independently	Tied to an outer class instance
Use Case	Utility/helper classes	Dependent on outer class behavior

When to Use Nested Classes?

- **Use a Static Inner Class When:**

- The inner class **does not depend** on the outer class instance
- The inner class is **only a utility/helper class**
- You want to reduce memory footprint by avoiding unnecessary references to the outer class

- **Use a Non-Static Inner Class When:**

- The inner class **needs to access non-static members** of the outer class
- The inner class is **closely associated** with the outer class logic
- The inner class represents **an integral part** of the outer class

Real-World Example:

- **Static Inner Class:** A `Logger` class inside a `Config` class that provides logging utilities
- **Non-Static Inner Class:** A `Button` class inside a `Window` class, where the button must always belong to a window

Lab work

